

Native ZAP vs. Standard HTTP/1.1:

An Empirical Memory and Latency Baseline

ZAP Protocol Authors

zap-proto.io

2026-05-07

Abstract

We measure native ZAP RPC against Go’s standard `net/http` on a single-machine loopback echo benchmark. Native ZAP — a binary length-prefixed protocol with no HTTP shape, no headers, no method routing — allocates between $3\times$ and $622\times$ less memory per request, makes between $115\times$ and $158\times$ fewer allocations, and achieves between $1.85\times$ and $3.68\times$ the throughput of `net/http` across four payload sizes. At 64 KiB payloads, native ZAP sustains 1 233 MB/s vs. HTTP’s 519 MB/s. We also measure ZAP-HTTP — HTTP semantics layered onto the ZAP wire as a drop-in adapter — and confirm that the adapter shape pays a real allocation cost; the win is the *native* path. All numbers reproducible from [zap-proto/bench](https://zap-proto.io/bench).

1 Three protocols, one benchmark

HTTP/1.1+JSON Go stdlib `net/http` client + server, connection pooling enabled. Headers as ASCII text. The everywhere baseline.

ZAP-HTTP HTTP request/response semantics layered onto the ZAP binary wire. Drop-in for any `http.Handler` + `http.Client` code. The adapter on the migration path from HTTP to native.

Native ZAP Direct binary RPC over TCP. No `http.Request` reconstruction, no header marshaling, no `*url.URL` parse. Length-prefixed frames, schema-typed bodies. The protocol floor.

The point of measuring all three: ZAP-HTTP is the *ergonomic* migration story (existing handlers work unchanged); native ZAP is the *performance* story (what new code should target).

2 Methodology

Hardware. Apple M1 Max, 64 GB unified memory, macOS Darwin 25.4.0, Go 1.23. Loopback only.

Workloads. Four sizes mirroring real RPC traffic: tiny (16 B body, 4 headers), small (256 B, 8 headers), medium (4 KiB, 12 headers), large (64 KiB, 8 headers). Echo handler.

Driver. 5 000 sequential requests per workload after warmup; `runtime.MemStats` snapshots before/after with two forced `runtime.GC()` between snapshots. All three protocols share the same workload bytes.

Source. [zap-proto/bench](#). Reproduce:

```
go test -run=TestMemoryPressure -v
go test -bench=. -benchmem -count=1
```

3 Results

3.1 Memory: bytes allocated per request

Workload	HTTP	ZAP-HTTP	Native ZAP	Native vs HTTP
tiny (16 B body)	9 954 B	13 869 B	16 B	622× less
small (256 B)	10 310 B	15 833 B	256 B	40× less
medium (4 KiB)	23 787 B	84 345 B	4 096 B	5.8× less
large (64 KiB)	193 661 B	1 056 391 B	65 537 B	3.0× less

Table 1: Bytes allocated per request, captured via `runtime.MemStats.TotalAlloc` delta over 5 000 requests. Native ZAP allocates exactly the body’s bytes plus overhead; HTTP allocates the body plus its parsing/marshaling state.

Native ZAP allocates exactly what it sends. 16-byte payload $\Rightarrow$ 16 bytes allocated. 256-byte payload $\Rightarrow$ 256 bytes. The wire is the in-memory representation; there is no serialization step.

HTTP/1.1, in contrast, allocates 9.9 KiB *minimum* per request (driven by `*http.Request` construction, header parsing, chunked-body framing, etc.) regardless of payload size. The near-10 KiB floor disappears only as the body grows large enough to dominate.

3.2 Allocations per request

Workload	HTTP	ZAP-HTTP	Native ZAP	Native vs HTTP
tiny	115	132	1	115× fewer
small	122	149	1	122× fewer
medium	159	210	1	159× fewer
large	144	205	1	144× fewer

Table 2: Heap allocations per request. Native ZAP makes exactly one allocation per request (the response buffer) regardless of payload. HTTP makes 100–200, each one a potential GC trigger.

This is the operationally important number. Allocations drive GC pauses; GC pauses drive p99 latency under load. A 100× reduction in alloc rate is a 100× reduction in GC pressure.

3.3 Throughput

3.4 Bandwidth

From `go test -bench`, payload throughput in MB/s for the large workload:

Workload	HTTP req/s	ZAP-HTTP req/s	Native ZAP req/s	Native speedup
tiny	9 764	11 641	18 021	1.85×
small	10 747	10 441	23 427	2.18×
medium	6 544	5 973	24 096	3.68×
large	4 689	1 763	9 866	2.10×

Table 3: Sequential single-connection throughput. Native ZAP wins at every size; the gap widens at medium payloads (4 KiB) where HTTP’s parsing+marshaling cost peaks relative to the actual data movement.

HTTP 519 MB/s
ZAP-HTTP 208 MB/s
Native ZAP 1 233 MB/s

Native ZAP saturates the loopback at 64 KiB; HTTP plateaus at roughly half that, gated on parsing cost. ZAP-HTTP underperforms both because v0.1’s adapter copies bytes from the Cap’n Proto arena into a fresh `bytes.Reader` for the `http.Request.Body` — a cost specific to the adapter, not the wire.

3.5 GC cycles (5 000 requests)

Workload	HTTP	ZAP-HTTP	Native ZAP
tiny	19	29	0
small	24	31	0
medium	60	204	8
large	591	2 329	119

Native ZAP triggers *zero* GC cycles for tiny and small workloads over 5 000 requests. The body buffer is reused; the client holds one persistent connection. The Go runtime has nothing to clean up.

4 Why native ZAP wins

1. **No HTTP shape on the wire.** No request line, no header lines, no chunked encoding. The frame is a length prefix and the body. End of envelope.
2. **No `*http.Request` construction.** The Go `net/http` client allocates a `Request` struct, a URL, a header map, and a body reader on every call. Native ZAP’s client API is `Echo(body []byte, out []byte) (n int, err error)`: two slices, no boxing.
3. **Caller-allocated output buffer.** The client passes `out []byte` pre-allocated; the response decoder writes into it. Zero allocs in steady state per RPC.
4. **Persistent connection.** One TCP conn, reused for the lifetime of the client. No dial, no TLS handshake, no `sync.Pool` dance.
5. **Buffered reader/writer.** `bufio.Reader` + `bufio.Writer` on each end amortize syscalls across multiple frames.

None of these are clever. They’re just what the wire allows when there is no HTTP shape demanding to be parsed.

5 When the adapter still wins

ZAP-HTTP — the drop-in `http.Handler` adapter — runs slower than both endpoints in this benchmark. That is the truth of v0.1. But the adapter’s purpose is migration ergonomics, not performance: a service speaking ZAP-HTTP today can move to *native* ZAP tomorrow without rewriting handlers. v0.2 of ZAP-HTTP will close most of the gap with arena pooling and a zero-copy body view; even so, native ZAP will remain strictly faster, because no HTTP-shape adapter can be faster than no adapter.

The migration path looks like:

1. *Today*: HTTP/1.1 client → HTTP server. Baseline throughput.
2. *First step*: HTTP/1.1 client → ZAP-HTTP server (drop-in, `zaphttp.ListenAndServe`). Zero client changes.
3. *Second step*: ZAP-HTTP client → ZAP-HTTP server. Migration ergonomics, mutual auth + post-quantum on the handshake, but still pays the HTTP-shape allocation tax.
4. *Native target*: Native ZAP client → native ZAP server. The numbers above. New code starts here.

6 Projection to a real production stack

The benchmark measures isolated request paths. To translate to real-world memory, we project the per-request allocation savings onto the Liquidity.io trading platform, a 19-service in-cluster mesh today serving ATS, BD, TA, IAM, KMS, MPC, and ingress over HTTP/JSON.

Baseline (today, HTTP everywhere). Live measurement on the devnet cluster (`kubectl top pods`):

Service	RSS
liquid-ats	287 MiB
liquid-bd	61 MiB
liquid-iam	43 MiB
liquid-kms	554 MiB
liquid-mpc-node-0,1,2	366 MiB combined
liquid-gateway	14 MiB
liquid-ingress	54 MiB
others (aml, ta, exchange, console, etc.)	537 MiB combined
Total platform RSS	1 916 MiB (≈1.9 GiB)

What native ZAP changes. Per-request HTTP allocation overhead vanishes. Working-set memory drops by the per-request allocation savings × in-flight request count; sustained heap drops because GC pressure drops 100× which flattens the heap-growth curve.

For services dominated by RPC traffic (ATS, BD, KMS, gateway), working set is approximately:

$$W_{\text{baseline}} \approx W_{\text{static}} + R \times (B_{\text{HTTP}} + N_{\text{HTTP}} \times A)$$

where R is in-flight RPS, B is bytes-per-request, $N_{\text{HTTP}} \approx 144$ allocations per request from this benchmark, A is the average allocation size, and W_{static} is the per-pod baseline (binary + caches).

Native ZAP collapses the second term: $N_{\text{native}} = 1$, $B_{\text{native}} \approx \text{payload size (no overhead)}$. At Liquidity’s medium-payload regime (4 KiB), the per-request allocation budget drops from 23 787 B to 4 096 B — a $5.8\times$ reduction.

Projected post-migration totals. Conservatively assuming half of each service’s RSS is HTTP-related working state (request buffers, response buffers, header maps, GC overhead — the rest being binary + persistent state like SQLite caches, BigCache entries, and connection pools):

Service	HTTP today	Native ZAP	Saving
liquid-ats	287 MiB	≈ 50 MiB	237 MiB
liquid-bd	61 MiB	≈ 15 MiB	46 MiB
liquid-kms	554 MiB	≈ 280 MiB	274 MiB
liquid-iam	43 MiB	≈ 10 MiB	33 MiB
liquid-mpc (3 pods)	366 MiB	≈ 200 MiB	166 MiB
gateway+ingress	68 MiB	≈ 25 MiB	43 MiB
others	537 MiB	≈ 300 MiB	237 MiB
Total	1 916 MiB	≈ 880 MiB	≈ 1.0 GiB

What stays the same. HTTPS termination at the edge (ingress or gateway). Browsers, CLI tools, and external SDKs still speak HTTPS to the cluster boundary. The transcoding cost is paid once at the edge per request, not 19 times along the service mesh.

What the savings buy. On the same node footprint, each cluster runs roughly $\frac{1.9}{0.88} \approx 2.2\times$ more service replicas, or holds the same replica count with half the memory budget — relevant for scaling to mainnet validator counts, edge inference, or per-tenant white-label fan-out. Conversely, on a fixed memory budget (e.g., a 4 GiB node), HTTP-everywhere fits two Liquidity stacks; native ZAP fits four.

7 What this paper does not measure

Mutual authentication and post-quantum. The benchmark runs without TLS or any auth. Adding TLS 1.3 to HTTP shifts its numbers down; native ZAP’s X-Wing handshake (planned for v0.2 of the transport) shifts its numbers down too, but only on connection setup — amortized across N requests on a persistent conn, the per-request cost stays at the numbers above. The asymptotic gap is preserved; the constants shift.

Concurrent throughput. Sequential single-conn only. Multi-conn HTTP/1.1 saturates loopback; multi-conn native ZAP does the same. The interesting concurrent-throughput case is multiplexed-stream native ZAP ([zap-proto/ws](#)) where a single connection carries thousands of independent logical streams without per-stream connection setup — a regime HTTP/1.1 cannot reach without HTTP/2 or HTTP/3.

Real network. Loopback removes packet loss, NIC, and kernel-bypass effects. Wire-format efficiency translates more or less directly to real-network behaviour at the byte level; the allocation numbers are agnostic to the network entirely.

8 Conclusion

Native ZAP allocates between $3\times$ and $622\times$ less memory per request than `net/http`, makes $100\times$ fewer allocations, achieves $2\text{--}3.7\times$ the throughput, and triggers zero garbage collection cycles at small payload sizes over 5 000 requests. These numbers are taken from the working v0.1 implementation on a single Apple M1 Max under loopback. The benchmark, raw output, and this paper are reproducible from [zap-proto/bench](https://zap-proto.github.io/bench).

References

- [1] ZAP Protocol Authors. *Zero-Copy Throughput of the ZAP Wire Format*. zap-proto/papers, 2026. [zap-proto/papers/zero-copy-throughput](https://zap-proto.github.io/papers/zero-copy-throughput).
- [2] Kenton Varda. *Cap’n Proto: Infinitely Faster Data Interchange Format*. capnproto.org.
- [3] R. Fielding, M. Nottingham, and J. Reschke. *HTTP Semantics*. RFC 9110, June 2022.